

AG35-CET&AG35-EUT

QuecOpen(SDK)音频开发指导

LTE Standard 模块系列

版本：1.0.1

日期：2025-03-27

状态：临时文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2025，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2025.

文档历史

本文档为临时版本，其中涉及的模块引脚或接口定义、频段、功能、特性及设计等（若有）发生变动的可能性较大，部分参数经过初步验证或仍未验证。变更或补充信息将在后续正式版本中体现。临时版本仅供临时参考使用，若与正式版本存在差异，应以最新版本为准。修订记录如下所示：

修订记录

版本	日期	作者	变更表述
-	2023-07-18	Joetar CHEN	文档创建
1.0.0	2023-07-18	Joetar CHEN	临时版本
1.0.1	2025-03-27	Joetar CHEN	临时版本 1. 基于 QuecOpen 方案统一命名，更新文档名称。 2. 新增适用模块 AG35-EUT。 3. 新增如下函数（第 2.3.53~2.3.54 章）： ql_audio_set_tone_enable_state() ql_audio_get_tone_enable_state()

目录

文档历史	3
目录	4
表格索引	6
1 引言	7
2 音频服务 API.....	8
2.1. 头文件.....	8
2.2. 函数概览.....	8
2.3. 函数详解.....	10
2.3.1. ql_audio_init	10
2.3.2. ql_audio_deinit	11
2.3.3. ql_audio_set_mixer_control	11
2.3.4. ql_audio_get_mixer_control	12
2.3.5. ql_audio_set_tx_voice_mic_gain	12
2.3.6. ql_audio_get_tx_voice_mic_gain	13
2.3.7. ql_audio_set_rx_voice_spkr_gain	14
2.3.8. ql_audio_get_rx_voice_spkr_gain	14
2.3.9. ql_audio_set_tx_voice_mute_state	15
2.3.10. ql_audio_get_tx_voice_mute_state	15
2.3.11. ql_audio_set_rx_voice_mute_state	16
2.3.12. ql_audio_get_rx_voice_mute_state	16
2.3.13. ql_audio_set_codec_up_vol	17
2.3.14. ql_audio_get_codec_up_vol	17
2.3.15. ql_audio_set_codec_down_vol.....	18
2.3.16. ql_audio_get_codec_down_vol.....	18
2.3.17. ql_audio_set_codec_mic_mute_state	19
2.3.18. ql_audio_get_codec_mic_mute_state	19
2.3.19. ql_audio_set_codec_spk_mute_state	20
2.3.20. ql_audio_get_codec_spk_mute_state	20
2.3.21. ql_audio_set_loopback_enable_state	21
2.3.22. ql_audio_get_loopback_enable_state	21
2.3.23. ql_audio_set_sidetone_gain	22
2.3.24. ql_audio_get_sidetone_gain	22
2.3.25. ql_audio_set_voice_call_manager_state.....	23
2.3.26. ql_audio_get_voice_call_manager_state	23
2.3.27. ql_audio_set_voice_stream_state	24
2.3.28. ql_audio_get_voice_stream_state	25
2.3.29. ql_audio_playback_open	25
2.3.29.1. QL_AUDIO_FE_PCM_DEV_E	26
2.3.29.2. QL_AUDIO_BE_DAI_E.....	26
2.3.29.3. 后端 DAI 掩码	27
2.3.30. ql_audio_playback_file_prepare	27

2.3.30.1.	ql_audio_pcm_config_t	28
2.3.30.2.	ql_audio_playback_state_cb_f.....	29
2.3.30.3.	QL_AUDIO_PLAYBACK_STATE_E.....	30
2.3.31.	ql_audio_playback_stream_prepare.....	30
2.3.32.	ql_audio_playback_play.....	31
2.3.33.	ql_audio_playback_push_stream	32
2.3.34.	ql_audio_playback_pause	33
2.3.35.	ql_audio_playback_resume	33
2.3.36.	ql_audio_playback_stop	33
2.3.37.	ql_audio_playback_close.....	34
2.3.38.	ql_audio_playback_get_state	35
2.3.39.	ql_audio_playback_set_block_flag	35
2.3.40.	ql_audio_capture_open	36
2.3.41.	ql_audio_capture_file_prepare	37
2.3.41.1.	ql_audio_capture_state_cb_f.....	38
2.3.41.2.	QL_AUDIO_CAPTURE_STATE_E	38
2.3.41.3.	QL_AUDIO_STREAM_FORMAT_E	39
2.3.42.	ql_audio_capture_stream_prepare.....	40
2.3.43.	ql_audio_capture_record	41
2.3.44.	ql_audio_capture_pull_stream.....	41
2.3.45.	ql_audio_capture_pause.....	42
2.3.46.	ql_audio_capture_resume	42
2.3.47.	ql_audio_capture_stop.....	43
2.3.48.	ql_audio_capture_close	43
2.3.49.	ql_audio_capture_get_state.....	44
2.3.50.	ql_audio_set_codec_switch	45
2.3.51.	ql_audio_get_codec_switch.....	45
2.3.52.	ql_audio_set_service_error_cb.....	46
2.3.52.1.	ql_audio_service_error_cb_f.....	46
2.3.53.	ql_audio_set_tone_enable_state	46
2.3.53.1.	ql_audio_service_tone_config_t	47
2.3.54.	ql_audio_get_tone_enable_state.....	48
3	示例	49
3.1.	播放音频文件实例	49
3.2.	捕获音频文件实例	51
3.3.	音频流边录边播实例.....	54
4	附录 参考文档及术语缩写	59

表格索引

表 1: 函数概览	8
表 2: 参考文档	59
表 3: 术语缩写	59

Preliminary
Quectel Confidential

1 引言

移远通信 AG35-CET 和 AG35-EUT 模块支持 QuecOpen®方案。QuecOpen®是基于 Linux 的嵌入式开发平台，可简化 IoV 应用的软件设计和开发过程。有关 QuecOpen 的详细信息，请参考文档 [1]。

本文档适用于 SDK 构建环境的 QuecOpen®方案，主要介绍 AG35-CET 和 AG35-EUT 模块在 QuecOpen®SDK 中提供的音频服务 API 及相关示例。

Preliminary
Quectel Confidential

2 音频服务 API

2.1. 头文件

音频服务 API 头文件为 `ql_audio_cfg.h` 和 `ql_audio_pcm.h`，位于 `ql-sysroots/usr/include/ql-sdk/` 目录下。若无特别说明，本文档所涉及头文件均在该目录下。

- `ql_audio_cfg.h` 中的 API 主要为配置接口，用于音频服务初始化、音量调节、静音设置等；
- `ql_audio_pcm.h` 中的 API 主要用于音频播放和捕获。

2.2. 函数概览

表 1：函数概览

函数	说明
<code>ql_audio_init()</code>	初始化音频服务
<code>ql_audio_deinit()</code>	去初始化音频服务
<code>ql_audio_set_mixer_control()</code>	设置 mixer 控件的值
<code>ql_audio_get_mixer_control()</code>	获取 mixer 控件的值
<code>ql_audio_set_tx_voice_mic_gain()</code>	设置语音通话上行链路的麦克风增益
<code>ql_audio_get_tx_voice_mic_gain()</code>	获取语音通话上行链路的麦克风增益
<code>ql_audio_set_rx_voice_spkr_gain()</code>	设置语音通话下行链路的扬声器增益
<code>ql_audio_get_rx_voice_spkr_gain()</code>	获取语音通话下行链路的扬声器增益
<code>ql_audio_set_tx_voice_mute_state()</code>	设置语音通话上行链路的静音状态
<code>ql_audio_get_tx_voice_mute_state()</code>	获取语音通话上行链路的静音状态
<code>ql_audio_set_rx_voice_mute_state()</code>	设置语音通话下行链路的静音状态

<code>ql_audio_get_rx_voice_mute_state()</code>	获取语音通话下行链路的静音状态
<code>ql_audio_set_codec_up_vol()</code>	设置 codec 的上行音量
<code>ql_audio_get_codec_up_vol()</code>	获取 codec 的上行音量
<code>ql_audio_set_codec_down_vol()</code>	设置 codec 的下行音量
<code>ql_audio_get_codec_down_vol()</code>	获取 codec 的下行音量
<code>ql_audio_set_codec_mic_mute_state()</code>	设置 codec 的麦克风静音状态
<code>ql_audio_get_codec_mic_mute_state()</code>	获取 codec 的麦克风静音状态
<code>ql_audio_set_codec_spk_mute_state()</code>	设置 codec 的扬声器静音状态
<code>ql_audio_get_codec_spk_mute_state()</code>	获取 codec 的扬声器静音状态
<code>ql_audio_set_loopback_enable_state()</code>	设置回环的使能状态
<code>ql_audio_get_loopback_enable_state()</code>	获取回环的使能状态
<code>ql_audio_set_sidetone_gain()</code>	设置侧音的增益
<code>ql_audio_get_sidetone_gain()</code>	获取侧音的增益
<code>ql_audio_set_voice_call_manager_state()</code>	设置语音通话服务的管理状态
<code>ql_audio_get_voice_call_manager_state()</code>	获取语音通话服务的管理状态
<code>ql_audio_set_voice_stream_state()</code>	设置语音流的状态
<code>ql_audio_get_voice_stream_state()</code>	获取语音流的状态
<code>ql_audio_playback_open()</code>	打开音频播放上下文
<code>ql_audio_playback_file_prepare()</code>	进行播放音频文件之前的准备工作
<code>ql_audio_playback_stream_prepare()</code>	进行播放音频流之前的准备工作
<code>ql_audio_playback_play()</code>	开始播放音频数据
<code>ql_audio_playback_push_stream()</code>	播放缓冲区中的音频流数据
<code>ql_audio_playback_pause()</code>	暂停音频播放
<code>ql_audio_playback_resume()</code>	恢复音频播放
<code>ql_audio_playback_stop()</code>	停止音频播放
<code>ql_audio_playback_close()</code>	关闭音频播放上下文
<code>ql_audio_playback_get_state()</code>	获取音频播放状态

<code>ql_audio_playback_set_block_flag()</code>	设置音频播放阻塞状态
<code>ql_audio_capture_open()</code>	打开音频捕获上下文
<code>ql_audio_capture_file_prepare()</code>	进行捕获音频文件之前的准备工作
<code>ql_audio_capture_stream_prepare()</code>	进行捕获音频流之前的准备工作
<code>ql_audio_capture_record()</code>	开始捕获音频数据
<code>ql_audio_capture_pull_stream()</code>	捕获音频流数据到缓冲区中
<code>ql_audio_capture_pause()</code>	暂停音频捕获
<code>ql_audio_capture_resume()</code>	恢复音频捕获
<code>ql_audio_capture_stop()</code>	停止音频捕获
<code>ql_audio_capture_close()</code>	关闭音频捕获上下文
<code>ql_audio_capture_get_state()</code>	获取当前的音频捕获状态
<code>ql_audio_set_codec_switch()</code>	切换模块内/外部 Codec
<code>ql_audio_get_codec_switch()</code>	获取当前使用的 Codec
<code>ql_audio_set_service_error_cb()</code>	设置音频服务异常回调函数
<code>ql_audio_set_tone_enable_state()</code>	设置提示音的播放状态
<code>ql_audio_get_tone_enable_state()</code>	获取提示音的播放状态

备注

若无特别说明，所有音频服务 API 均不支持并发调用。

2.3. 函数详解

2.3.1. ql_audio_init

该函数用于初始化音频服务。

- 函数原型

```
int ql_audio_init(void)
```

- 参数

无

- 返回值

`QL_ERR_OK` 函数执行成功

`QL_ERR_SERVICE_NOT_READY` 函数执行失败，建议重试。

其他值 函数执行失败，错误码详见 `ql_type.h`。

备注

使用其他音频服务接口前，必须调用本函数初始化音频服务。

2.3.2. ql_audio_deinit

该函数用于去初始化音频服务。

- 函数原型

```
int ql_audio_deinit(void)
```

- 参数

无

- 返回值

`QL_ERR_OK` 函数执行成功

其他值 函数执行失败，错误码详见 `ql_type.h`。

2.3.3. ql_audio_set_mixer_control

该函数用于设置 mixer 控件的值。

- 函数原型

```
int ql_audio_set_mixer_control(const char *control, const char *val_list)
```

- 参数

control:

[In] mixer 控件的名称。

val_list:

[In] 待设置的 mixer 控件的值；用字符串表示，多个值之间用空格隔开。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化音频服务。
QL_ERR_SERVICE_ABORT	函数执行失败，音频服务出错。
QL_ERR_INVALID_ARG	函数执行失败，非法参数。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

2.3.4. ql_audio_get_mixer_control

该函数用于获取 mixer 控件的值。

- 函数原型

```
int ql_audio_get_mixer_control(const char *control, char *val_list_buf, uint32_t buf_size)
```

- 参数

control:

[In] mixer 控件的名称。

val_list_buf:

[Out] 存放 mixer 控件值的缓冲区。

buf_size:

[In] 缓冲区大小；单位：字节。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化音频服务。
QL_ERR_SERVICE_ABORT	函数执行失败，音频服务出错。
QL_ERR_INVALID_ARG	函数执行失败，非法参数。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

2.3.5. ql_audio_set_tx_voice_mic_gain

该函数用于设置语音通话上行链路的麦克风增益。

- 函数原型

```
int ql_audio_set_tx_voice_mic_gain(int32_t mic_gain)
```

- 参数

mic_gain:

[In] 待设置的麦克风增益；取值范围：0~65535。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化音频服务。
QL_ERR_SERVICE_ABORT	函数执行失败，音频服务出错。
QL_ERR_INVALID_ARG	函数执行失败，非法参数。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

备注

该函数须在打开语音流之前调用，否则设置无法生效。

2.3.6. ql_audio_get_tx_voice_mic_gain

该函数用于获取语音通话上行链路的麦克风增益。

- 函数原型

```
int ql_audio_get_tx_voice_mic_gain(int32_t *p_mic_gain)
```

- 参数

p_mic_gain:

[Out] 当前语音通话上行链路的麦克风增益。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化音频服务。
QL_ERR_SERVICE_ABORT	函数执行失败，音频服务出错。
QL_ERR_INVALID_ARG	函数执行失败，非法参数。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

2.3.7. ql_audio_set_rx_voice_spkr_gain

该函数用于设置语音通话下行链路的扬声器增益。

- 函数原型

```
int ql_audio_set_rx_voice_spkr_gain(int32_t spkr_gain)
```

- 参数

spkr_gain:

[In] 待设置的扬声器增益；取值范围：0~65535。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化音频服务。
QL_ERR_SERVICE_ABORT	函数执行失败，音频服务出错。
QL_ERR_INVALID_ARG	函数执行失败，非法参数。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

备注

该函数须在打开语音流之前调用，否则设置无法生效。

2.3.8. ql_audio_get_rx_voice_spkr_gain

该函数用于获取语音通话下行链路的扬声器增益。

- 函数原型

```
int ql_audio_get_rx_voice_spkr_gain(int32_t *p_spkr_gain)
```

- 参数

p_spkr_gain:

[Out] 当前语音通话下行链路的扬声器增益。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化音频服务。
QL_ERR_SERVICE_ABORT	函数执行失败，音频服务出错。
QL_ERR_INVALID_ARG	函数执行失败，非法参数。

其他值

函数执行失败，错误码详见 *ql_type.h*。

2.3.9. ql_audio_set_tx_voice_mute_state

该函数用于设置语音通话上行链路的静音状态。

● 函数原型

```
int ql_audio_set_tx_voice_mute_state(int32_t mute_state)
```

● 参数

mute_state:

[In] 待设置的静音状态。

QL_AUDIO_STATE_1	静音
QL_AUDIO_STATE_0	不静音

● 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化音频服务。
QL_ERR_SERVICE_ABORT	函数执行失败，音频服务出错。
QL_ERR_INVALID_ARG	函数执行失败，非法参数。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

备注

该接口函数须在打开语音流之后调用，否则设置无法生效。

2.3.10. ql_audio_get_tx_voice_mute_state

该函数用于获取语音通话上行链路的静音状态。

● 函数原型

```
int ql_audio_get_tx_voice_mute_state(int32_t *p_mute_state)
```

● 参数

p_mute_state:

[Out] 当前语音通话上行链路的静音状态。

QL_AUDIO_STATE_1	静音
QL_AUDIO_STATE_0	不静音

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化音频服务。
QL_ERR_SERVICE_ABORT	函数执行失败，音频服务出错。
QL_ERR_INVALID_ARG	函数执行失败，非法参数。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

2.3.11. ql_audio_set_rx_voice_mute_state

该函数用于设置语音通话下行链路的静音状态。

- 函数原型

```
int ql_audio_set_rx_voice_mute_state(int32_t mute_state)
```

- 参数

mute_state:

[In] 待设置的静音状态。

QL_AUDIO_STATE_1	静音
QL_AUDIO_STATE_0	不静音

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化音频服务。
QL_ERR_SERVICE_ABORT	函数执行失败，音频服务出错。
QL_ERR_INVALID_ARG	函数执行失败，非法参数。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

备注

该接口函数须在打开语音流之后调用，否则设置无法生效。

2.3.12. ql_audio_get_rx_voice_mute_state

该函数用于获取语音通话下行链路的静音状态。

- 函数原型

```
int ql_audio_get_rx_voice_mute_state(int32_t *p_mute_state)
```

- 参数

p_mute_state:

[Out] 当前语音通话下行链路的静音状态。

QL_AUDIO_STATE_1	静音
QL_AUDIO_STATE_0	不静音

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化音频服务。
QL_ERR_SERVICE_ABORT	函数执行失败，音频服务出错。
QL_ERR_INVALID_ARG	函数执行失败，非法参数。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

2.3.13. ql_audio_set_codec_up_vol

该函数用于设置 codec 的上行音量。

- 函数原型

```
int ql_audio_set_codec_up_vol(int32_t up_volume)
```

- 参数

up_volume:

[In] 待设置的上行音量；取值范围：0~100。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化音频服务。
QL_ERR_SERVICE_ABORT	函数执行失败，音频服务出错。
QL_ERR_INVALID_ARG	函数执行失败，非法参数。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

2.3.14. ql_audio_get_codec_up_vol

该函数用于获取 codec 的上行音量。

- 函数原型

```
int ql_audio_get_codec_up_vol(int32_t *p_up_volume)
```

- 参数

p_up_volume:

[Out] 当前 codec 的上行音量。

- 返回值

<i>QL_ERR_OK</i>	函数执行成功
<i>QL_ERR_NOT_INIT</i>	函数执行失败，未初始化音频服务。
<i>QL_ERR_SERVICE_ABORT</i>	函数执行失败，音频服务出错。
<i>QL_ERR_INVALID_ARG</i>	函数执行失败，非法参数。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

2.3.15. ql_audio_set_codec_down_vol

该函数用于设置 codec 的下行音量。

- 函数原型

```
int ql_audio_set_codec_down_vol(int32_t down_volume)
```

- 参数

down_volume:

[In] 待设置的下行音量；取值范围：0~100。

- 返回值

<i>QL_ERR_OK</i>	函数执行成功
<i>QL_ERR_NOT_INIT</i>	函数执行失败，未初始化音频服务。
<i>QL_ERR_SERVICE_ABORT</i>	函数执行失败，音频服务出错。
<i>QL_ERR_INVALID_ARG</i>	函数执行失败，非法参数。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

2.3.16. ql_audio_get_codec_down_vol

该函数用于获取 codec 的下行音量。

- 函数原型

```
int ql_audio_get_codec_down_vol(int32_t *p_down_volume)
```

- 参数

p_down_volume:

[Out] 当前 codec 的下行音量。

- 返回值

<i>QL_ERR_OK</i>	函数执行成功
<i>QL_ERR_NOT_INIT</i>	函数执行失败，未初始化音频服务。
<i>QL_ERR_SERVICE_ABORT</i>	函数执行失败，音频服务出错。
<i>QL_ERR_INVALID_ARG</i>	函数执行失败，非法参数。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

2.3.17. ql_audio_set_codec_mic_mute_state

该函数用于设置 codec 的麦克风静音状态。

- 函数原型

```
int ql_audio_set_codec_mic_mute_state(int32_t mute_state)
```

- 参数

mute_state:

[In] 待设置的静音状态。

<i>QL_AUDIO_STATE_1</i>	静音
<i>QL_AUDIO_STATE_0</i>	不静音

- 返回值

<i>QL_ERR_OK</i>	函数执行成功
<i>QL_ERR_NOT_INIT</i>	函数执行失败，未初始化音频服务。
<i>QL_ERR_SERVICE_ABORT</i>	函数执行失败，音频服务出错。
<i>QL_ERR_INVALID_ARG</i>	函数执行失败，非法参数。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

2.3.18. ql_audio_get_codec_mic_mute_state

该函数用于获取 codec 的麦克风静音状态。

- 函数原型

```
int ql_audio_get_codec_mic_mute_state(int32_t *p_mute_state)
```

- 参数

p_mute_state:

[Out] 当前 codec 的麦克风静音状态。

QL_AUDIO_STATE_1 静音
QL_AUDIO_STATE_0 不静音

- 返回值

QL_ERR_OK 函数执行成功
QL_ERR_NOT_INIT 函数执行失败，未初始化音频服务。
QL_ERR_SERVICE_ABORT 函数执行失败，音频服务出错。
QL_ERR_INVALID_ARG 函数执行失败，非法参数。
其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.19. ql_audio_set_codec_spk_mute_state

该函数用于设置 codec 的扬声器静音状态。

- 函数原型

```
int ql_audio_set_codec_spk_mute_state(int32_t mute_state)
```

- 参数

mute_state:

[In] 待设置的静音状态。

QL_AUDIO_STATE_1 静音
QL_AUDIO_STATE_0 不静音

- 返回值

QL_ERR_OK 函数执行成功
QL_ERR_NOT_INIT 函数执行失败，未初始化音频服务。
QL_ERR_SERVICE_ABORT 函数执行失败，音频服务出错。
QL_ERR_INVALID_ARG 函数执行失败，非法参数。
其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.20. ql_audio_get_codec_spk_mute_state

该函数用于获取 codec 的扬声器静音状态。

- 函数原型

```
int ql_audio_get_codec_spk_mute_state(int32_t *p_mute_state)
```

- 参数

p_mute_state:

[Out] 当前 codec 的扬声器静音状态。

QL_AUDIO_STATE_1	静音
QL_AUDIO_STATE_0	不静音

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化音频服务。
QL_ERR_SERVICE_ABORT	函数执行失败，音频服务出错。
QL_ERR_INVALID_ARG	函数执行失败，非法参数。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

2.3.21. ql_audio_set_loopback_enable_state

该函数用于设置回环的使能状态。

- 函数原型

```
int ql_audio_set_loopback_enable_state(int32_t enable_state)
```

- 参数

enable_state:

[In] 待设置的回环状态。

QL_AUDIO_STATE_1	使能回环功能
QL_AUDIO_STATE_0	禁用回环功能

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化音频服务。
QL_ERR_SERVICE_ABORT	函数执行失败，音频服务出错。
QL_ERR_INVALID_ARG	函数执行失败，非法参数。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

2.3.22. ql_audio_get_loopback_enable_state

该函数用于获取回环的使能状态。

- 函数原型

```
int ql_audio_get_loopback_enable_state(int32_t *p_enable_state)
```

- 参数

p_enable_state:

[Out] 当前回环的使能状态。

QL_AUDIO_STATE_1	回环功能已使能
QL_AUDIO_STATE_0	回环功能已禁用

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化音频服务。
QL_ERR_SERVICE_ABORT	函数执行失败，音频服务出错。
QL_ERR_INVALID_ARG	函数执行失败，非法参数。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

2.3.23. ql_audio_set_sidetone_gain

该函数用于设置侧音增益。

- 函数原型

```
int ql_audio_set_sidetone_gain(int32_t sidetone_gain)
```

- 参数

sidetone_gain:

[In] 待设置的侧音增益；取值范围：0~65535。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化音频服务。
QL_ERR_SERVICE_ABORT	函数执行失败，音频服务出错。
QL_ERR_INVALID_ARG	函数执行失败，非法参数。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

2.3.24. ql_audio_get_sidetone_gain

该函数用于获取侧音增益。

- 函数原型

```
int ql_audio_get_sidetone_gain(int32_t *p_sidetone_gain)
```

- 参数

p_sidetone_gain:

[Out] 当前的侧音增益。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化音频服务。
QL_ERR_SERVICE_ABORT	函数执行失败，音频服务出错。
QL_ERR_INVALID_ARG	函数执行失败，非法参数。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

2.3.25. ql_audio_set_voice_call_manager_state

该函数用于设置语音通话服务的管理状态。默认情况下，来电铃声、回铃声、第三方来电提示音以及语音流的开启和关闭等语音通话服务由进程 *ql_audiod* 实现。

- 函数原型

```
int ql_audio_set_voice_call_manager_state(int32_t manager_state)
```

- 参数

manager_state:

[In] 待设置的管理状态。语音通话服务包括：来电铃声、回铃声、第三方来电提示音以及语音流的开启或关闭等。

QL_AUDIO_STATE_1	由用户自行实现所述语音通话服务
QL_AUDIO_STATE_0	由服务程序 <i>ql_audiod</i> 实现所述语音通话服务

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化音频服务。
QL_ERR_SERVICE_ABORT	函数执行失败，音频服务出错。
QL_ERR_INVALID_ARG	函数执行失败，非法参数。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

2.3.26. ql_audio_get_voice_call_manager_state

该函数用于获取语音通话服务的管理状态。

- 函数原型

```
int ql_audio_get_voice_call_manager_state(int32_t *p_manager_state)
```

● 参数

p_manager_state:

[Out] 当前语音通话服务的管理状态。

<code>QL_AUDIO_STATE_1</code>	由用户自行实现所述语音通话服务
<code>QL_AUDIO_STATE_0</code>	由服务程序 <code>ql_audiod</code> 实现所述语音通话服务

● 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_NOT_INIT</code>	函数执行失败，未初始化音频服务。
<code>QL_ERR_SERVICE_ABORT</code>	函数执行失败，音频服务出错。
<code>QL_ERR_INVALID_ARG</code>	函数执行失败，非法参数。
其他值	函数执行失败，错误码详见 <code>ql_type.h</code> 。

2.3.27. ql_audio_set_voice_stream_state

该函数用于设置语音流的状态。

● 函数原型

```
int ql_audio_set_voice_stream_state(int32_t stream_state)
```

● 参数

stream_state:

[In] 待设置的语音流状态。

<code>QL_AUDIO_STATE_1</code>	打开语音流
<code>QL_AUDIO_STATE_0</code>	关闭语音流

● 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_NOT_INIT</code>	函数执行失败，未初始化音频服务。
<code>QL_ERR_SERVICE_ABORT</code>	函数执行失败，音频服务出错。
<code>QL_ERR_INVALID_ARG</code>	函数执行失败，非法参数。
其他值	函数执行失败，错误码详见 <code>ql_type.h</code> 。

备注

调用接口函数 `ql_audio_set_voice_call_manager_state()` 设置管理状态为 `QL_AUDIO_STATE_1` 后，建立语音通话时，服务程序 `ql_audiod` 将不会打开语音流；此时，调用 `ql_audio_set_voice_stream_state()` 设置语音流状态为 `QL_AUDIO_STATE_1` 即可打开语音流，再设置语音流状态为 `QL_AUDIO_STATE_0` 即可关闭语音流。

2.3.28. ql_audio_get_voice_stream_state

该函数用于获取语音流的状态。

● 函数原型

```
int ql_audio_get_voice_stream_state(int32_t *p_stream_state)
```

● 参数

p_stream_state:

[Out] 当前语音流的状态。

QL_AUDIO_STATE_1	语音流打开
QL_AUDIO_STATE_0	语音流关闭

● 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化音频服务。
QL_ERR_SERVICE_ABORT	函数执行失败，音频服务出错。
QL_ERR_INVALID_ARG	函数执行失败，非法参数。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

2.3.29. ql_audio_playback_open

该函数用于打开音频播放上下文。

● 函数原型

```
ql_audio_handle_t ql_audio_playback_open(QL_AUDIO_FE_PCM_DEV_E fe_pcm_dev,
                                          uint32_t be_dai_mask)
```

● 参数

fe_pcm_dev:

[In] 前端 PCM 设备类型；详见第 2.3.29.1 章。

be_dai_mask:

[In] 后端 DAI 掩码；取值定义详见第 2.3.29.3 章，现支持如下类型：

QL_AUDIO_BE_DAI_MASK_PLAYBACK_PRI_PCM	将音频数据播放到第一路 PCM 接口
QL_AUDIO_BE_DAI_MASK_PLAYBACK_VOICE_TX	将音频数据播放到语音通话上行链路

● 返回值

QL_AUDIO_INVALID_HANDLE	函数执行失败，无效句柄。
返回一个有效句柄	函数执行成功

2.3.29.1. QL_AUDIO_FE_PCM_DEV_E

前端 PCM 设备类型，枚举信息定义如下：

```
typedef enum QL_AUDIO_FE_PCM_DEV_ENUM
{
    QL_AUDIO_FE_PCM_DEV_MIN = -1,
    QL_AUDIO_FE_PCM_DEV_MULTIMEDIA1 = 0,
    QL_AUDIO_FE_PCM_DEV_MULTIMEDIA2,
    QL_AUDIO_FE_PCM_DEV_MULTIMEDIA3,
    QL_AUDIO_FE_PCM_DEV_MAX
} QL_AUDIO_FE_PCM_DEV_E
```

● 成员

成员	描述
QL_AUDIO_FE_PCM_DEV_MULTIMEDIA1	一般音频播放、捕获可用的第一个 PCM 设备
QL_AUDIO_FE_PCM_DEV_MULTIMEDIA2	一般音频播放、捕获可用的第二个 PCM 设备
QL_AUDIO_FE_PCM_DEV_MULTIMEDIA3	一般音频播放、捕获可用的第三个 PCM 设备

2.3.29.2. QL_AUDIO_BE_DAI_E

后端 DAI 类型，枚举信息定义如下：

```
typedef enum QL_AUDIO_BE_DAI_ENUM
{
    QL_AUDIO_BE_DAI_MIN = -1,
    QL_AUDIO_BE_DAI_PLAYBACK_PRI_PCM = 0,
    QL_AUDIO_BE_DAI_PLAYBACK_VOICE_TX,
    QL_AUDIO_BE_DAI_CAPTURE_PRI_PCM,
    QL_AUDIO_BE_DAI_CAPTURE_VOICE_UL,
    QL_AUDIO_BE_DAI_CAPTURE_VOICE_DL,
    QL_AUDIO_BE_DAI_MAX
} QL_AUDIO_BE_DAI_E
```

● 成员

成员	描述
QL_AUDIO_BE_DAI_PLAYBACK_PRI_PCM	将音频播放到第一路 PCM 接口
QL_AUDIO_BE_DAI_PLAYBACK_VOICE_TX	将音频播放到语音通话上行链路

QL_AUDIO_BE_DAI_CAPTURE_PRI_PCM	从第一路 PCM 接口捕获音频
QL_AUDIO_BE_DAI_CAPTURE_VOICE_UL	从语音通话上行链路捕获语音流
QL_AUDIO_BE_DAI_CAPTURE_VOICE_DL	从语音通话下行链路捕获语音流

2.3.29.3. 后端 DAI 掩码

```
#define QL_AUDIO_BE_DAI_MASK_PLAYBACK_PRI_PCM (1 << QL_AUDIO_BE_DAI_PLAYBACK_PRI_PCM)
#define QL_AUDIO_BE_DAI_MASK_PLAYBACK_VOICE_TX (1 << QL_AUDIO_BE_DAI_PLAYBACK_VOICE_TX)
#define QL_AUDIO_BE_DAI_MASK_CAPTURE_PRI_PCM (1 << QL_AUDIO_BE_DAI_CAPTURE_PRI_PCM)
#define QL_AUDIO_BE_DAI_MASK_CAPTURE_VOICE_UL (1 << QL_AUDIO_BE_DAI_CAPTURE_VOICE_UL)
#define QL_AUDIO_BE_DAI_MASK_CAPTURE_VOICE_DL (1 << QL_AUDIO_BE_DAI_CAPTURE_VOICE_DL)
```

备注

DAI 掩码是根据第 2.3.29.2 章中的 DAI 类型得到的；掩码可按位或进行操作，比如，期望将音频同时播放到第一路 PCM 接口和语音通话上行链路中，可进行 `QL_AUDIO_BE_DAI_MASK_PLAYBACK_PRI_PCM|QL_AUDIO_BE_DAI_MASK_PLAYBACK_VOICE_TX` 操作。

2.3.30. ql_audio_playback_file_prepare

该函数用于进行播放音频文件之前的准备工作。

- 函数原型

```
int ql_audio_playback_file_prepare(ql_audio_handle_t handle,
                                   const char *file_name,
                                   ql_audio_pcm_config_t *pcm_config,
                                   ql_audio_playback_state_cb_f playback_state_cb,
                                   void *params)
```

- 参数

handle:

[In] 由函数 `ql_audio_playback_open()` 返回的句柄。

file_name:

[In] 待播放的文件名。

pcm_config:

[In] PCM 配置参数；详见第 2.3.30.1 章。一般指定为 NULL，表示采用默认值。对于 .wav 格式的文件，PCM 配置参数默认值可从文件头中获取；对于其他类型的文件，默认值如下：

<i>period_size</i>	0
<i>period_count</i>	1
<i>num_channels</i>	1
<i>sample_rate</i>	8000
<i>pcm_format</i>	2

playback_state_cb:

[In] 音频播放状态回调函数，上报当前的播放状态；详见第 2.3.30.2 章。

params:

[In] 回调函数携带的参数。

● 返回值

QL_ERR_OK	函数执行成功
QL_ERR_INVALID_HANDLE	函数执行失败，无效句柄。
QL_ERR_INVALID_STATE	函数执行失败，无效状态。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

备注

1. 调用该函数之前，必须先调用 *ql_audio_playback_open()*。
2. 若期望播放音频文件，则需先调用该函数做好播放音频文件之前的准备工作，再调用 *ql_audio_playback_play()* 开始播放音频数据。

2.3.30.1. ql_audio_pcm_config_t

PCM 配置参数，结构体定义如下：

```
typedef struct ql_audio_pcm_config_struct
{
    uint32_t period_size;
    uint32_t period_count;
    uint32_t num_channels;
    uint32_t sample_rate;
    uint32_t pcm_format;
} ql_audio_pcm_config_t
```

● 参数

类型	参数	说明
uint32_t	<i>period_size</i>	硬件缓冲区 period 大小。默认 320 字节。
uint32_t	<i>period_count</i>	应用程序从硬件缓冲区读取的数据所占用的 period 个数，或者向硬件缓冲区写入的数据所占用的 period 个数。 取值范围 1~2, <i>sample_rate</i> 为 8000 Hz 时, <i>period_count</i> 为 1; <i>sample_rate</i> 为 16000 Hz 时, <i>period_count</i> 为 2。
uint32_t	<i>num_channels</i>	声道数; 1 表示单声道, 仅支持单声道
uint32_t	<i>sample_rate</i>	采样率; PCM 接口支持 8000 和 16000, 单位: Hz
uint32_t	<i>pcm_format</i>	PCM 数据格式; 目前仅支持取值 2, 表示 16 bit 小端格式

2.3.30.2.ql_audio_playback_state_cb_f

播放状态回调函数，定义如下：

```
typedef int (*ql_audio_playback_state_cb_f)(ql_audio_handle_t handle,
                                             void *params,
                                             QL_AUDIO_PLAYBACK_STATE_E state)
```

● 参数

handle:

[In] 播放句柄; 为 *ql_audio_playback_open()* 的返回值。

params:

[In] 回调函数携带的参数。

state:

[In] 当前的音频播放状态; 音频播放状态详见第 2.3.30.3 章。

● 返回值

用户自定义的返回值

备注

不可在回调函数中调用其它 API 函数。

2.3.30.3. QL_AUDIO_PLAYBACK_STATE_E

音频播放状态，枚举信息定义如下：

```
typedef enum QL_AUDIO_PLAYBACK_STATE_ENUM
{
    QL_AUDIO_PLAYBACK_STATE_CLOSE = 0,
    QL_AUDIO_PLAYBACK_STATE_OPEN,
    QL_AUDIO_PLAYBACK_STATE_PREPARE,
    QL_AUDIO_PLAYBACK_STATE_PLAYING,
    QL_AUDIO_PLAYBACK_STATE_FINISHED,
    QL_AUDIO_PLAYBACK_STATE_PAUSE,
    QL_AUDIO_PLAYBACK_STATE_ERROR,
} QL_AUDIO_PLAYBACK_STATE_E
```

● 成员

成员	描述
QL_AUDIO_PLAYBACK_STATE_CLOSE	关闭状态
QL_AUDIO_PLAYBACK_STATE_OPEN	打开状态
QL_AUDIO_PLAYBACK_STATE_PREPARE	就绪状态
QL_AUDIO_PLAYBACK_STATE_PLAYING	播放状态
QL_AUDIO_PLAYBACK_STATE_FINISHED	完成状态
QL_AUDIO_PLAYBACK_STATE_PAUSE	暂停状态
QL_AUDIO_PLAYBACK_STATE_ERROR	出错状态

2.3.31. ql_audio_playback_stream_prepare

该函数用于进行播放音频流之前的准备工作。

● 函数原型

```
int ql_audio_playback_stream_prepare(ql_audio_handle_t handle,
                                     ql_audio_pcm_config_t *pcm_config,
                                     ql_audio_playback_state_cb_f playback_state_cb,
                                     void *params)
```

- 参数

handle:

[In] 由函数 `ql_audio_playback_open()` 返回的句柄。

pcm_config:

[In] PCM 配置参数；详见第 2.3.30.1 章。若该参数指定为 NULL，则按默认值处理。默认值如下：

<i>period_size</i>	0
<i>period_count</i>	1
<i>num_channels</i>	1
<i>sample_rate</i>	8000
<i>pcm_format</i>	2

playback_state_cb:

[In] 音频播放状态回调函数，上报当前的播放状态；详见第 2.3.30.2 章。

params:

[In] 回调函数携带的参数。

- 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_INVALID_HANDLE</code>	函数执行失败，无效句柄。
<code>QL_ERR_INVALID_STATE</code>	函数执行失败，无效状态。
其他值	函数执行失败，错误码详见 <code>ql_type.h</code> 。

备注

1. 调用该函数之前，必须先调用 `ql_audio_playback_open()`。
2. 若期望播放一段音频流，则需先调用该函数做好播放音频流之前的准备工作，再调用 `ql_audio_playback_push_stream()` 开始播放音频流数据。

2.3.32. ql_audio_playback_play

该函数用于开始播放音频数据。

- 函数原型

```
int ql_audio_playback_play(ql_audio_handle_t handle)
```

- 参数

handle:

[In] 由函数 `ql_audio_playback_open()` 返回的句柄。

● 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_INVALID_HANDLE</code>	函数执行失败，无效句柄。
<code>QL_ERR_INVALID_STATE</code>	函数执行失败，无效状态。
其他值	函数执行失败，错误码详见 <code>ql_type.h</code> 。

备注

1. 调用该函数之前，必须先调用 `ql_audio_playback_file_prepare()`准备好待播放的音频文件，方可实现音频文件中的音频数据的播放。
2. 该函数亦可实现音频流数据的播放。用于音频流数据播放时，应在调用该函数之前，先调用 `ql_audio_playback_stream_prepare()` 准备好待播放的音频流，最后再调用 `ql_audio_playback_push_stream()`实现音频流数据的播放。

2.3.33. ql_audio_playback_push_stream

该函数用于播放缓冲区中的音频流数据。

● 函数原型

```
int ql_audio_playback_push_stream(ql_audio_handle_t handle,
                                  void *stream_buf,
                                  uint32_t buf_size)
```

● 参数

handle:

[In] 由函数 `ql_audio_playback_open()`返回的句柄。

stream_buf:

[In] 存放待播放音频流数据的缓冲区。

buf_size:

[In] 待播放的音频流数据大小。单位：字节。

● 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_INVALID_HANDLE</code>	函数执行失败，无效句柄；
其他值	函数执行失败，错误码详见 <code>ql_type.h</code> 。

2.3.34. ql_audio_playback_pause

该函数用于暂停音频播放。

- 函数原型

```
int ql_audio_playback_pause(ql_audio_handle_t handle)
```

- 参数

handle:

[In] 由函数 *ql_audio_playback_open()*返回的句柄。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_INVALID_HANDLE	函数执行失败，无效句柄。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

2.3.35. ql_audio_playback_resume

该函数用于恢复音频播放。

- 函数原型

```
int ql_audio_playback_resume(ql_audio_handle_t handle)
```

- 参数

handle:

[In] 由函数 *ql_audio_playback_open()*返回的句柄。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_INVALID_HANDLE	函数执行失败，无效句柄。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

2.3.36. ql_audio_playback_stop

该函数用于停止音频播放。

- 函数原型

```
int ql_audio_playback_stop(ql_audio_handle_t handle)
```

- 参数

handle:

[In] 由函数 `ql_audio_playback_open()` 返回的句柄。

- 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_INVALID_HANDLE</code>	函数执行失败，无效句柄。
其他值	函数执行失败，错误码详见 <code>ql_type.h</code> 。

备注

若音频处于播放状态或者暂停状态，调用该函数都会结束音频播放，且无法恢复。

2.3.37. ql_audio_playback_close

该函数用于关闭音频播放上下文。

- 函数原型

```
int ql_audio_playback_close(ql_audio_handle_t handle)
```

- 参数

handle:

[In] 由函数 `ql_audio_playback_open()` 返回的句柄。

- 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_INVALID_HANDLE</code>	函数执行失败，无效句柄。
其他值	函数执行失败，错误码详见 <code>ql_type.h</code> 。

备注

音频播放结束后，必须调用该函数关闭音频播放上下文，否则会影响后续 `ql_audio_playback_open()` 的正常调用。

2.3.38. ql_audio_playback_get_state

该函数用于获取音频播放状态。

● 函数原型

```
int ql_audio_playback_get_state(ql_audio_handle_t handle,
                                QL_AUDIO_PLAYBACK_STATE_E *playback_state)
```

● 参数

handle:

[In] 由函数 *ql_audio_playback_open()* 返回的句柄。

playback_state:

[Out] 当前的音频播放状态；详见第 2.3.30.3 章。

● 返回值

QL_ERR_OK	函数执行成功
QL_ERR_INVALID_HANDLE	函数执行失败，无效句柄。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

2.3.39. ql_audio_playback_set_block_flag

该函数用于设置音频播放阻塞状态。

● 函数原型

```
int ql_audio_playback_set_block_flag(ql_audio_handle_t handle, uint8_t flags)
```

● 参数

handle:

[In] 由函数 *ql_audio_playback_open()* 返回的句柄。

flags:

[In] 阻塞标志。

QL_AUDIO_PLAYBACK_BLOCK	阻塞
QL_AUDIO_PLAYBACK_NONBLOCK	非阻塞

● 返回值

QL_ERR_OK	函数执行成功
QL_ERR_INVALID_HANDLE	函数执行失败，无效句柄。
QL_ERR_INVALID_ARG	函数执行失败，无效参数。

QL_ERR_INVALID_STATE
其他值

函数执行失败，无效状态。
函数执行失败，错误码详见 *ql_type.h*。

备注

1. 如需通过 *ql_audio_playback_set_block_flag()* 实现音频播放阻塞，则该函数的调用应符合如下流程方可生效：
 - 1) *ql_audio_playback_open()*
 - 2) *ql_audio_playback_set_block_flag()*
 - 3) *ql_audio_playback_file_prepare()*
 - 4) *ql_audio_playback_play()*
2. 通过 *ql_audio_playback_set_block_flag()* 成功实现音频播放阻塞后：
 - 1) 音频文件播放过程中，若拨打电话或者有电话来电，则音频文件播放暂停；电话挂断后，音频文件将继续播放；
 - 2) 但语音通话过程中，无法调用函数实现音频播放；此时 *ql_audio_playback_file_prepare()* 函数调用会失败，因此无法播放音频文件。

2.3.40. ql_audio_capture_open

该函数用于打开音频捕获上下文。

● 函数原型

```
ql_audio_handle_t ql_audio_capture_open(QL_AUDIO_FE_PCM_DEV_E fe_pcm_dev,
                                         uint32_t be_dai_mask)
```

● 参数

fe_pcm_dev:

[In] 前端 PCM 设备类型；详见第 2.3.29.1 章。

be_dai_mask:

[In] 后端 DAI 掩码；取值定义详见第 2.3.29.3 章，现支持如下类型：

QL_AUDIO_BE_DAI_MASK_CAPTURE_PRI_PCM	从第一路 PCM 接口捕获音频数据
QL_AUDIO_BE_DAI_MASK_CAPTURE_VOICE_UL	从语音通话上行链路捕获语音数据
QL_AUDIO_BE_DAI_MASK_CAPTURE_VOICE_DL	从语音通话下行链路捕获语音数据

● 返回值

QL_AUDIO_INVALID_HANDLE
返回一个有效句柄

函数执行失败，无效句柄。
函数执行成功

2.3.41. ql_audio_capture_file_prepare

该函数用于进行捕获音频文件之前的准备工作。

● 函数原型

```
int ql_audio_capture_file_prepare(ql_audio_handle_t handle,
                                const char *file_name,
                                QL_AUDIO_STREAM_FORMAT_E type,
                                ql_audio_pcm_config_t *pcm_config,
                                ql_audio_capture_state_cb_f capture_state_cb,
                                void *params)
```

● 参数

handle:

[In] 由函数 *ql_audio_capture_open()* 返回的句柄。

file_name:

[In] 待捕获的音频文件名。

type:

[In] 音频文件中音频数据的格式；详见第 2.3.41.3 章；
现仅支持 *QL_AUDIO_STREAM_FORMAT_PCM*。

pcm_config:

[In] PCM 配置参数；详见第 2.3.30.1 章。若该参数指定为 NULL，则按默认值处理。默认值如下：

<i>period_size</i>	0
<i>period_count</i>	2
<i>num_channels</i>	1
<i>sample_rate</i>	8000
<i>pcm_format</i>	2

capture_state_cb:

[In] 音频捕获状态回调函数，上报当前的捕获状态；详见第 2.3.41.1 章。

params:

[In] 回调函数携带的参数。

● 返回值

<i>QL_ERR_OK</i>	函数执行成功
<i>QL_ERR_INVALID_HANDLE</i>	函数执行失败，无效句柄；
<i>QL_ERR_INVALID_STATE</i>	函数执行失败，无效状态；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

备注

1. 调用该函数之前，必须先调用 `ql_audio_capture_open()`。
2. 若期望捕获音频文件，则需先调用该函数做好捕获音频文件之前的准备工作，再调用 `ql_audio_capture_record()` 开始捕获音频数据。

2.3.41.1. ql_audio_capture_state_cb_f

捕获状态回调函数，定义如下：

```
typedef int (*ql_audio_capture_state_cb_f)(ql_audio_handle_t handle,
void *params,
QL_AUDIO_CAPTURE_STATE_E state)
```

● 参数

handle:

[In] 播放句柄；为 `ql_audio_capture_open()` 的返回值。

params:

[In] 回调函数携带的参数。

state:

[In] 当前的音频捕获状态；音频捕获状态详见第 2.3.41.2 章。

● 返回值

用户自定义的返回值

备注

不可在回调函数中调用其它 API 函数。

2.3.41.2. QL_AUDIO_CAPTURE_STATE_E

音频捕获状态，枚举信息定义如下：

```
typedef enum QL_AUDIO_CAPTURE_STATE_ENUM
{
    QL_AUDIO_CAPTURE_STATE_CLOSE = 0,
    QL_AUDIO_CAPTURE_STATE_OPEN,
    QL_AUDIO_CAPTURE_STATE_PREPARE,
```

```
QL_AUDIO_CAPTURE_STATE_CAPTURING,
QL_AUDIO_CAPTURE_STATE_FINISHED,
QL_AUDIO_CAPTURE_STATE_PAUSE,
QL_AUDIO_CAPTURE_STATE_ERROR,
} QL_AUDIO_CAPTURE_STATE_E
```

● 成员

成员	描述
QL_AUDIO_CAPTURE_STATE_CLOSE	关闭状态
QL_AUDIO_CAPTURE_STATE_OPEN	打开状态
QL_AUDIO_CAPTURE_STATE_PREPARE	就绪状态
QL_AUDIO_CAPTURE_STATE_CAPTURING	捕获状态
QL_AUDIO_CAPTURE_STATE_FINISHED	完成状态
QL_AUDIO_CAPTURE_STATE_PAUSE	暂停状态
QL_AUDIO_CAPTURE_STATE_ERROR	出错状态

2.3.41.3. QL_AUDIO_STREAM_FORMAT_E

音频流格式，枚举信息定义如下：

```
typedef enum QL_AUDIO_STREAM_FORMAT_ENUM
{
    QL_AUDIO_STREAM_FORMAT_PCM = 1,
    QL_AUDIO_STREAM_FORMAT_MP3,
    QL_AUDIO_STREAM_FORMAT_AMR,
    QL_AUDIO_STREAM_FORMAT_AMRNB,
    QL_AUDIO_STREAM_FORMAT_AMRWB,
} QL_AUDIO_STREAM_FORMAT_E
```

● 成员

成员	描述
QL_AUDIO_STREAM_FORMAT_PCM	PCM 格式
QL_AUDIO_STREAM_FORMAT_MP3	MP3 格式

<code>QL_AUDIO_STREAM_FORMAT_AMR</code>	AMR 格式
<code>QL_AUDIO_STREAM_FORMAT_AMRNB</code>	AMR-NB 格式
<code>QL_AUDIO_STREAM_FORMAT_AMRWB</code>	AMR-WB 格式

2.3.42. `ql_audio_capture_stream_prepare`

该函数用于进行捕获音频流之前的准备工作。

● 函数原型

```
int ql_audio_capture_stream_prepare(ql_audio_handle_t handle,
                                   ql_audio_pcm_config_t *pcm_config,
                                   ql_audio_capture_state_cb_f capture_state_cb,
                                   void *params)
```

● 参数

handle:

[In] 由函数 `ql_audio_capture_open()` 返回的句柄。

pcm_config:

[In] PCM 配置参数；详见第 2.3.30.1 章。若该参数指定为 NULL，则按默认值处理。默认值如下：

<i>period_size</i>	0
<i>period_count</i>	2
<i>num_channels</i>	1
<i>sample_rate</i>	8000
<i>pcm_format</i>	2

capture_state_cb:

[In] 音频捕获状态回调函数，上报当前的捕获状态；详见第 2.3.41.1 章。

params:

[In] 回调函数携带的参数。

● 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_INVALID_HANDLE</code>	函数执行失败，无效句柄。
<code>QL_ERR_INVALID_STATE</code>	函数执行失败，无效状态。
其他值	函数执行失败，错误码详见 <code>ql_type.h</code> 。

备注

1. 调用该函数之前，必须先调用 `ql_audio_capture_open()`。
2. 若期望捕获一段音频流，则需先调用该函数做好捕获音频流之前的准备工作，再调用 `ql_audio_capture_push_stream()`开始捕获音频流数据。

2.3.43. ql_audio_capture_record

该函数用于开始捕获音频数据。

● 函数原型

```
int ql_audio_capture_record(ql_audio_handle_t handle)
```

● 参数

handle:

[In] 由函数 `ql_audio_capture_open()`返回的句柄。

● 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_INVALID_HANDLE</code>	函数执行失败，无效句柄。
<code>QL_ERR_INVALID_STATE</code>	函数执行失败，无效状态。
其他值	函数执行失败，错误码详见 <code>ql_type.h</code> 。

备注

1. 调用该函数之前，必须先调用 `ql_audio_capture_file_prepare()`为音频文件的捕获做好准备，方可实现音频文件的捕获和保存。
2. 该函数亦可实现音频流数据的捕获。用于音频流数据捕获时，应在调用该函数之前，先调用 `ql_audio_capture_stream_prepare()` 为音频流的捕获做好准备，最后再调用 `ql_audio_capture_pull_stream()`实现音频流数据的捕获。

2.3.44. ql_audio_capture_pull_stream

该函数用于捕获音频流数据到缓冲区中。

● 函数原型

```
int ql_audio_capture_pull_stream(ql_audio_handle_t handle,
                                void *stream_buf,
                                uint32_t buf_size)
```

- 参数

handle:

[In] 由函数 `ql_audio_capture_open()` 返回的句柄。

stream_buf:

[Out] 存放待捕获音频流数据的缓冲区。

buf_size:

[In] 缓冲区大小；单位：字节。

- 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_INVALID_HANDLE</code>	函数执行失败，无效句柄。
其他值	函数执行失败，错误码详见 <code>ql_type.h</code> 。

2.3.45. ql_audio_capture_pause

该函数用于暂停音频捕获。

- 函数原型

```
int ql_audio_capture_pause(ql_audio_handle_t handle)
```

- 参数

handle:

[In] 由函数 `ql_audio_capture_open()` 返回的句柄。

- 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_INVALID_HANDLE</code>	函数执行失败，无效句柄。
其他值	函数执行失败，错误码详见 <code>ql_type.h</code> 。

2.3.46. ql_audio_capture_resume

该函数用于恢复音频捕获。

- 函数原型

```
int ql_audio_capture_resume(ql_audio_handle_t handle)
```

- 参数

handle:

[In] 由函数 *ql_audio_capture_open()*返回的句柄。

- 返回值

<i>QL_ERR_OK</i>	函数执行成功
<i>QL_ERR_INVALID_HANDLE</i>	函数执行失败，无效句柄。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

2.3.47. ql_audio_capture_stop

该函数用于停止音频捕获。

- 函数原型

```
int ql_audio_capture_stop(ql_audio_handle_t handle)
```

- 参数

handle:

[In] 由函数 *ql_audio_capture_open()*返回的句柄。

- 返回值

<i>QL_ERR_OK</i>	函数执行成功
<i>QL_ERR_INVALID_HANDLE</i>	函数执行失败，无效句柄。
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

备注

若音频处于捕获状态或者暂停状态，调用该函数将会结束音频捕获，且无法恢复。

2.3.48. ql_audio_capture_close

该函数用于关闭音频捕获上下文。

- 函数原型

```
int ql_audio_capture_close(ql_audio_handle_t handle)
```

- 参数

handle:

[In] 由函数 `ql_audio_capture_open()` 返回的句柄。

- 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_INVALID_HANDLE</code>	函数执行失败，无效句柄。
其他值	函数执行失败，错误码详见 <code>ql_type.h</code> 。

备注

音频捕获结束后，必须调用该函数关闭音频捕获上下文，否则会影响后续 `ql_audio_capture_open()` 的正常调用。

2.3.49. ql_audio_capture_get_state

该函数用于获取当前的音频捕获状态。

- 函数原型

```
int ql_audio_capture_get_state(ql_audio_handle_t handle,
                               QL_AUDIO_CAPTURE_STATE_E *capture_state)
```

- 参数

handle:

[In] 由函数 `ql_audio_capture_open()` 返回的句柄。

capture_state:

[Out] 当前的音频捕获状态；详见第 2.3.41.2 章。

- 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_INVALID_HANDLE</code>	函数执行失败，无效句柄。
其他值	函数执行失败，错误码详见 <code>ql_type.h</code> 。

2.3.50. ql_audio_set_codec_switch

该函数用于切换模块内/外部 Codec。

- 函数原型

```
int ql_audio_set_codec_switch(int32_t codec_switch)
```

- 参数

codec_switch:

[In] 选择内部 Codec 或外部 Codec。

0 外部 Codec

1 内部 Codec

- 返回值

QL_ERR_OK

函数执行成功

QL_ERR_NOT_INIT

函数执行失败，未初始化音频服务。

QL_ERR_SERVICE_ABORT

函数执行失败，音频服务出错。

备注

模块有内置 Codec，但默认使用外部 Codec。外部 Codec 可调整音量，静音状态时，内部 Codec 无法调整音量。

2.3.51. ql_audio_get_codec_switch

该函数用于获取当前使用的 Codec。

- 函数原型

```
int ql_audio_get_codec_switch(int32_t *p_codec_switch)
```

- 参数

p_codec_switch:

[Out] 当前使用的内/外部 codec 类型。

0 外部 codec

1 内部 codec

- 返回值

QL_ERR_OK

函数执行成功

QL_ERR_NOT_INIT 函数执行失败，未初始化音频服务。
QL_ERR_SERVICE_ABORT 函数执行失败，音频服务出错。

2.3.52. ql_audio_set_service_error_cb

该函数用于设置音频服务异常回调函数。

- 函数原型

```
int ql_audio_set_service_error_cb(ql_audio_service_error_cb_f cb)
```

- 参数

cb:

[In] 音频服务异常回调函数；详见第2.3.52.1章。只有当音频服务异常退出时，才会执行回调函数。

- 返回值

QL_ERR_OK 函数执行成功
QL_ERR_NOT_INIT 函数执行失败，未初始化音频服务。
QL_ERR_SERVICE_ABORT 函数执行失败，音频服务出错。

2.3.52.1. ql_audio_service_error_cb_f

音频服务异常回调函数，定义如下：

```
typedef void (*ql_audio_service_error_cb_f)(int error)
```

- 参数

error:

[In] 错误码，详见 ql_type.h。

- 返回值

用户自定义的返回值。

2.3.53. ql_audio_set_tone_enable_state

该函数用于设置提示音的播放状态。

● 函数原型

```
int ql_audio_set_tone_enable_state(int32_t enable_state,
                                   ql_audio_tone_config_t *p_tone_config);
```

● 参数

enable_state:

[In] 待设置的提示音播放状态。

QL_AUDIO_STATE_1 开始播放提示音
QL_AUDIO_STATE_0 停止播放提示音

p_tone_config:

[In] 待设置的提示音配置参数；详见第2.3.53.1章。

enable_state 为 *QL_AUDIO_STATE_1* 时，该参数必须指定；
enable_state 为 *QL_AUDIO_STATE_0* 时，该参数为 NULL。

● 返回值

QL_ERR_OK 函数执行成功
 其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.53.1.ql_audio_tone_config_t

提示音配置参数结构体定义如下：

```
typedef struct ql_audio_tone_config_struct
{
    int32_t low_fre;
    int32_t high_fre;
    int32_t on_duration;
    int32_t off_duration;
    int32_t volume;
    int32_t count;
}ql_audio_tone_config_t;
```

● 参数

类型	参数	描述
int32_t	<i>low_fre</i>	低频。范围：100~4000；单位：Hz
int32_t	<i>high_fre</i>	高频。范围：100~4000；单位：Hz
int32_t	<i>on_duration</i>	播放时长。单位：毫秒

int32_t	<i>off_duration</i>	相邻两次播放的时间间隔。单位：毫秒
int32_t	<i>volume</i>	播放音量大小。范围：0~100
int32_t	<i>count</i>	播放次数

2.3.54. ql_audio_get_tone_enable_state

该函数用于获取提示音的播放状态。

- 函数原型

```
int ql_audio_get_tone_enable_state(int32_t *p_enable_state);
```

- 参数

p_enable_state:

[Out] 当前提示音的播放状态。

QL_AUDIO_STATE_1 开始播放提示音
QL_AUDIO_STATE_0 停止播放提示音

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败，错误码详见 *ql_type.h*。

3 示例

本章所述代码示例均摘自 `sample/audio/main.c`，此文件下可查看音频服务 API 函数的完整示例。本章主要介绍部分示例，以演示如何使用播放/录音接口实现音频播放和捕获。

3.1. 播放音频文件实例

```

/** 定义一个音频文件播放结束标志 */
static int g_palyback_end = 0;

/** 定义音频初始化最大重试次数 */
#define AUDIO_INIT_MAX_TRY_CNT 100

/** 定义音频文件播放状态回调函数 */
int playback_state_cb (ql_audio_handle_t handle,
                      void *params,
                      QL_AUDIO_PLAYBACK_STATE_E state)
{
    /** 音频文件播放状态处理 */
    switch (state)
    {
        /** 当音频文件播放结束或者出错，将结束标志设置为 1 */
        case QL_AUDIO_PLAYBACK_STATE_FINISHED:
        case QL_AUDIO_PLAYBACK_STATE_ERROR:
        {
            g_playback_end = 1;
            break;
        }
    }

    return 0;
}

int main()
{
    int ret = QL_ERR_OK;

```

```

int retry_cnt = 0;
int audio_is_init = 0;
ql_audio_handle_t playback_handle = QL_AUDIO_INVALID_HANDLE;

/** 初始化音频服务 */
while (AUDIO_INIT_MAX_TRY_CNT > retry_cnt)
{
    ret = ql_audio_init();
    if (QL_ERR_SERVICE_NOT_READY == ret)
    {
        printf("audio service is not ready, try again, try count = %d\n",
               (retry_cnt + 1));
        usleep(200 * 1000);
        retry_cnt++;
    }
    else if (QL_ERR_OK == ret)
    {
        printf("Success to initialize audio service\n");
        audio_is_init = 1;
        break;
    }
    else
    {
        printf("Failed to initialize audio service, ret = %d\n", ret);
        break;
    }
}

if (1 != audio_is_init)
{
    printf("Failed to initialize audio service\n");
    return -1;
}

/** 打开音频播放上下文 */
playback_handle = ql_audio_playback_open(QL_AUDIO_FE_PCM_DEV_MULTIMEDIA1,
                                           QL_AUDIO_BE_DAI_MASK_PLAYBACK_PRI_PCM);
if (QL_AUDIO_INVALID_HANDLE == playback_handle)
{
    /** 若返回无效句柄，应返回-1 */
    return -1;
}

/** 设置音频文件播放阻塞标志，也可不设置 */

```

```

ql_audio_playback_set_block_flag(playback_handle, QL_AUDIO_PLAYBACK_BLOCK);

/** 音频文件播放准备 */
ret = ql_audio_playback_file_prepare(playback_handle, "/etc/ringtone1.wav", NULL,
                                     playback_state_cb, NULL);

if (QL_ERR_OK != ret)
{
    /** 若接口调用失败，必须调用 close 接口 */
    ql_audio_playback_close(playback_handle);
    return -1;
}

/** 开始播放音频文件 */
ret = ql_audio_playback_play(playback_handle);
{
    /** 若接口调用失败，必须调用 close 接口 */
    ql_audio_playback_close(playback_handle);
    return -1;
}

while (0 == g_playback_end)
{
    /** 循环等待结束标志,若 g_playback_end 设置为 1，则结束循环 */
    usleep(100 * 1000);
}

/** 音频文件播放结束后，必须调用 close 接口 */
ql_audio_playback_close(playback_handle);
return 0;
}

```

3.2. 捕获音频文件实例

```

/** 定义一个音频文件捕获结束标志 */
static int g_capture_end = 0;

/** 定义音频初始化最大重试次数 */
#define AUDIO_INIT_MAX_TRY_CNT 100

```

```

/** 音频捕获状态回调函数 */
int capture_state_cb (ql_audio_handle_t handle, void *params,
                      QL_AUDIO_CAPTURE_STATE_E state)
{
    /** 音频捕获状态处理 */
    switch (state)
    {
        /** 当捕获结束或者出错，将结束标志设置为 1 */
        case QL_AUDIO_CAPTURE_STATE_FINISHED:
        case QL_AUDIO_CAPTURE_STATE_ERROR:
        {
            g_capture_end = 1;
            break;
        }
    }

    return 0;
}

int main()
{
    int ret = QL_ERR_OK;
    int retry_cnt = 0;
    int audio_is_init = 0;
    ql_audio_handle_t capture_handle = QL_AUDIO_INVALID_HANDLE;
    ql_audio_pcm_config_t pcm_config;

    /** 初始化音频服务 */
    while (AUDIO_INIT_MAX_TRY_CNT > retry_cnt)
    {
        ret = ql_audio_init();
        if (QL_ERR_SERVICE_NOT_READY == ret)
        {
            printf("audio service is not ready, try again, try count = %d\n",
                   (retry_cnt + 1));
            usleep(200 * 1000);
            retry_cnt++;
        }
        else if (QL_ERR_OK == ret)
        {
            printf("Success to initialize audio service\n");
            audio_is_init = 1;
            break;
        }
    }
}

```

```

        else
        {
            printf("Failed to initialize audio service, ret = %d\n", ret);;
            break;
        }
    }

    if (1 != audio_is_init)
    {
        printf("Failed to initialize audio service\n");
        return -1;
    }

    /** 打开音频捕获上下文 */
    capture_handle = ql_audio_capture_open(QL_AUDIO_FE_PCM_DEV_MULTIMEDIA1,
                                           QL_AUDIO_BE_DAI_MASK_CAPTURE_PRI_PCM);
    if (QL_AUDIO_INVALID_HANDLE == capture_handle)
    {
        /** 若返回无效句柄，应返回-1 */
        return -1;
    }

    /** 定义 PCM 参数，也可不定义，若不定义，则按默认值处理 */
    memset(&pcm_config, 0, sizeof(ql_audio_pcm_config_t));
    pcm_config.period_size = 320;
    pcm_config.period_count = 2;
    pcm_config.num_channels = 1;
    pcm_config.sample_rate = 8000;
    pcm_config.format = 2;

    /** 音频文件捕获准备 */
    ret = ql_audio_capture_file_prepare(capture_handle, "/data/record.wav",
                                       QL_AUDIO_STREAM_FORMAT_PCM,
                                       &pcm_config, capture_state_cb, NULL);
    if (QL_ERR_OK != ret)
    {
        /** 若接口调用失败，必须调用 close 接口 */
        ql_audio_capture_close(capture_handle);
        return -1;
    }

    /** 开始捕获音频文件 */
    ret = ql_audio_capture_record(capture_handle);
    {

```

```

    /** 若接口调用失败，必须调用 close 接口 */
    ql_audio_capture_close(capture_handle);
    return -1;
}

while (0 == g_capture_end)
{
    /** 循环等待结束标志，若 g_capture_end 设置为 1，则结束循环 */
    usleep(100* 1000);
}

/** 音频捕获结束后，必须调用 close 接口 */
ql_audio_capture_close(capture_handle);
return 0;
}

```

3.3. 音频流边录边播实例

```

/** 定义音频流捕获和播放结束标志 */
static int g_playback_end = 0;
static int g_cpature_end = 0;

/** 定义音频初始化最大重试次数 */
#define AUDIO_INIT_MAX_TRY_CNT 100

/** 音频流播放状态回调函数 */
int playback_stream_state_cb (ql_audio_handle_t handle, void *params,
                             QL_AUDIO_PLAYBACK_STATE_E state)
{
    /** 音频流播放状态处理 */
    switch (state)
    {
        /** 当音频流播放出错，将结束标志设置为 1 */
        case QL_AUDIO_PLAYBACK_STATE_ERROR:
        {
            g_playback_end = 1;
            break;
        }
    }
}

return 0;

```

```

}

/** 音频流捕获状态回调函数 */
int capture_stream_state_cb (ql_audio_handle_t handle, void *params,
                             QL_AUDIO_CAPTURE_STATE_E state)
{
    /** 音频流捕获状态处理 */
    switch (state)
    {
        /** 当音频流捕获出错，将结束标志设置为 1 */
        case QL_AUDIO_CAPTURE_STATE_ERROR:
        {
            g_cpature_end = 1;
            break;
        }
    }

    return 0;
}

int main()
{
    int ret = QL_ERR_OK;
    int retry_cnt = 0;
    int audio_is_init = 0;
    char stream_buf[640] = { 0 };
    ql_audio_handle_t playback_handle = QL_AUDIO_INVALID_HANDLE;
    ql_audio_handle_t captue_handle = QL_AUDIO_INVALID_HANDLE;
    ql_audio_pcm_config_t pcm_config;

    /** 初始化音频服务 */
    while (AUDIO_INIT_MAX_TRY_CNT > retry_cnt)
    {
        ret = ql_audio_init();
        if (QL_ERR_SERVICE_NOT_READY == ret)
        {
            printf("audio service is not ready, try again, try count = %d\n",
                   (retry_cnt + 1));
            usleep(200 * 1000);
            retry_cnt++;
        }
        else if (QL_ERR_OK == ret)
        {
            printf("Success to initialize audio service\n");
        }
    }
}

```

```

        audio_is_init = 1;
        break;
    }
    else
    {
        printf("Failed to initialize audio service, ret = %d\n", ret);
        break;
    }
}

if (1 != audio_is_init)
{
    printf("Failed to initialize audio service\n");
    return -1;
}

/** 打开音频播放上下文 */
playback_handle = ql_audio_playback_open(QL_AUDIO_FE_PCM_DEV_MULTIMEDIA1,
                                           QL_AUDIO_BE_DAI_MASK_PLAYBACK_PRI_PCM);
if (QL_AUDIO_INVALID_HANDLE == playback_handle)
{
    /** 若返回无效句柄，应返回-1 */
    return -1;
}

/** 定义 PCM 参数 */
memset(&pcm_config, 0, sizeof(ql_audio_pcm_config_t));
pcm_config.period_size = 320;
pcm_config.period_count = 2;
pcm_config.num_channels = 1;
pcm_config.sample_rate = 8000;
pcm_config.format = 2;

/** 音频播放准备 */
ret = ql_audio_playback_stream_prepare(playback_handle, &pcm_config,
                                       playback_stream_state_cb, NULL);

if (QL_ERR_OK != ret)
{
    /** 若接口调用失败，必须调用 close 接口 */
    ql_audio_playback_close(playback_handle);
    return -1;
}

/** 开始播放音频流 */

```

```

ret = ql_audio_playback_play(playback_handle);
if (QL_ERR_OK != ret)
{
    /** 若接口调用失败，必须调用 close 接口 */
    ql_audio_playback_close(playback_handle);
    return -1;
}

/** 打开音频捕获上下文 */
capture_handle = ql_audio_capture_open(QL_AUDIO_FE_PCM_DEV_MULTIMEDIA1,
                                       QL_AUDIO_BE_DAI_MASK_CAPTURE_PRI_PCM);
if (QL_AUDIO_INVALID_HANDLE == capture_handle)
{
    /** 若返回无效句柄，应返回-1 */
    ql_audio_playback_close(playback_handle);
    return -1;
}

/** 定义 PCM 参数 */
memset(&pcm_config, 0, sizeof(ql_audio_pcm_config_t));
pcm_config.period_size = 320;
pcm_config.period_count = 2;
pcm_config.num_channels = 1;
pcm_config.sample_rate = 8000;
pcm_config.format = 2;

/** 音频流捕获准备 */
ret = ql_audio_capture_stream_prepare(capture_handle, &pcm_config, NULL, NULL);
if (QL_ERR_OK != ret)
{
    /** 若接口调用失败，必须调用 close 接口 */
    ql_audio_capture_close(capture_handle);
    ql_audio_playback_close(playback_handle);
    return -1;
}

/** 开始捕获音频 */
ret = ql_audio_capture_record(capture_handle);
{
    /** 若接口调用失败，必须调用 close 接口 */
    ql_audio_capture_close(capture_handle);
    ql_audio_playback_close(playback_handle);
    return -1;
}

```

```

while ((0 == g_cpature_end) && (0 == g_playback_end))
{
    memset(stream_buf, 0, sizeof(stream_buf));
    ql_audio_capture_pull_stream(capture_handle, stream_buf, 320 * 2);
    ql_audio_playback_push_stream(capture_handle, stream_buf, 320 * 2);
}

/** 结束后，必须调用函数关闭音频上下文 */
ql_audio_capture_close(capture_handle);
ql_audio_playback_close(playback_handle);
return 0;
}

```

Preliminary
Quectel Confidential

4 附录 参考文档及术语缩写

表 2：参考文档

文档名称
[1] Quectel_AG35-CET&AG35-EUT_QuecOpen(SDK)_快速开发指导

表 3：术语缩写

缩写	英文全称	中文全称
AMR	Adaptive Multi-Rate compression	自适应多速率音频压缩
AMR-NB	Adaptive Multi Rate-Narrow Band Speech Codec	自适应多速率窄带
AMR-WB	Adaptive Multi-Rate Wideband	自适应多速率宽带
API	Application Programming Interface	应用程序编程接口
DAI	Digital Audio Interface	数字音频接口
DSP	Digital Signal Processor	数字信号处理器
I2S	Inter-IC Sound	集成电路内置音频总线
IoV	Internet of Vehicles	车联网
MP3	Moving Picture Experts Group Audio Layer III	动态影像专家压缩标准音频层面 3
PCM	Pulse Code Modulation	脉冲编码调制
SDK	Software Development Kit	软件开发工具包